

ゲームプログラム 仕様・設計説明資料

本資料では、本ゲームにおけるメインロジック（主に `GameScene` 周辺）のプログラム設計、各種システムの実装手法について解説します。

1. ゲームループと状態管理 (ステートマシン)

ゲームのメイン処理を担う `GameScene` クラスでは、現在のゲーム状況を `GameState` という列挙型(enum)を用いて管理しています。

```
enum GameState {  
    Playing,      // プレイ中  
    Paused,       // 一時停止中  
    PlayerHit,    // 被弾演出中  
    GameClear,    // クリア  
    GameOver,     // ゲームオーバー  
    Editor        // ステージ編集モード  
};
```

毎フレーム実行される `Update()` メソッド内でこの `GameState` を判定し、`PlayingUpdate()` や `PausedUpdate()` などの専用関数に処理を振り分けています。これにより、「ポーズ画面中に裏でプレイヤーが動いてしまう」といったバグを防ぎ、処理の見通しを良くしています。

2. 主なゲームシステム群

プレイヤーとオブジェクト管理

操作キャラクターの処理は `Player` クラスにカプセル化されています。敵や障害物などの配置物は `GameObjectManager` が一括で保持・更

当たり判定処理 (Collision)

`CheckCollisions()` メソッドによって、プレイヤーと敵・障害物・アイテム等との衝突を毎フレーム計算しています。

新・描画の指示を出しています。
これにより、画面上に無数のオブジェクトが存在しても管理ループは常にクリーンに保たれます。

被弾時には状態が直ちに `PlayerHit` へ遷移し、ゲームオーバーへの演出が開始されます。

🌟 演出・パーティクル (Particles)

視覚的なフィードバックを強化するため、`GameSceneParticle` マネージャーを導入しています。オブジェクトの破壊時やアイテム取得時など、特定のアクションに合わせてパーティクルを生成し、画面を彩ります。

📖 ステージとエディター連携

ステージの構造や難易度設定は `StageSettings` クラスが担います。また、内部に `EditorUI` を組み込んでおり、`scene.json` を直接読み書きして開発中（あるいはゲーム内のEditor状態）にリアルタイムでステージを調整できる仕組みを持っています。

3. スコア・ランキングシステムの実装

プレイヤーの進行度や成果を評価するため、スコアと走行距離の2軸でランキングを管理しています。

- **走行距離 (Distance):** `currentDistance_` 変数で進行した距離を追跡し、上位3位を `topRankings_[3]` の配列に記録します。
- **スコア (Score):** 敵を倒した数やアイテム取得による点数を `currentScore_` として計算。連続で敵を倒した際の `bonusEnemyHitCount_` (コンボボーナス) もスコアリングに影響を与え、上位3位を `topScoreRankings_[3]` に保持します。

ゲームオーバーやクリア時に `UpdateRanking()` や `UpdateScoreRanking()` が呼ばれ、配列内の数値をソート・更新するシンプルな構造でランキングを実現しています。

4. ポーズ（一時停止）システムの独立

ゲームを中断する処理は、ゲームの進行状況とは切り離す必要があるため `PauseSystem` クラスとして完全に独立させています。状態が `Paused` のときは `PauseSystem` のUIや入力を

受け付け、オブジェクト群のUpdateをスキップすることで「ゲームの時間が止まっている」状態を作ります。

設計のポイント: 単一の巨大なクラス (神クラス) を作るのではなく、`Player`、`GameObjectManager`、`GameSceneParticle`、`StageSettings` など役割ごとにクラスを小さく切り分けています。これにより、今後の機能追加やバグ修正が非常に容易になっています。